第 10 章：文件与数据持久化

文件处理的核心不是“会调用 `read()` 和 `write()`”，而是把内存对象安全地变成可长期保存、可被其他程序理解的字节，并在读取时把不可信数据逐层还原和校验。

程序退出后，普通变量会消失；文件则可以把配置、日志、导入导出数据和缓存留到下一次运行。本章将建立一条完整的数据流：路径定位、资源管理、文本编解码、格式解析、业务校验，以及失败时如何保护旧数据。

学完本章，你应该能做什么

- 使用 `pathlib.Path` 表示路径，并区分“路径”与“已打开的文件”；
- 使用 `with` 管理文件生命周期，理解它与 Java try-with-resources 的相同点和边界；
- 明确选择打开模式与文本编码，并根据文件大小选择整体读取或流式读取；
- 区分 JSON/CSV 的格式解析、字段类型转换与业务校验；
- 使用同目录临时文件和原子替换降低配置文件写坏的风险；
- 解释为什么 pickle 只能用于可信数据；
- 根据互操作性、安全性、数据结构和数据规模选择存储方案。

先建立整体心智模型

读取文件并不是一步完成的。对于来自磁盘或外部系统的数据，可以把处理过程理解成下面这条管道：

```
路径 → 打开文件 → 读取字节 → 按编码得到文本 → 按格式解析 → 转换字段类型 → 校验业务规则
 Path    file object       UTF-8       JSON / CSV       int / bool       ID、范围、唯一性
```

每一层解决不同问题。例如，JSON 解析成功只说明括号和引号合法，并不说明 `id` 是正整数；CSV 解析器能识别带逗号的字段，却不会自动把字符串 `"false"` 变成布尔值。

写入则大致沿相反方向进行：

```
业务对象 → 可序列化结构 → JSON/CSV 文本 → UTF-8 字节 → 文件
```

Java 工程师可以先用下面的对照定位概念：

Java 工程师可以先用下面的对照定位概念：

关注点	Java 中常见做法	Python 中常见做法	不能完全等同之处
路径	`java.nio.file.Path`	`pathlib.Path`	两者都只是路径表示，不代表文件已经打开
资源释放	try-with-resources	`with` 上下文管理器	`with` 是通用协议，不只管理实现 `Closeable` 的对象
文本编码	`StandardCharsets.UTF_8`	`encoding="utf-8"`	Python 省略编码时会使用环境默认值，不适合可移植数据
JSON	Jackson、Gson	标准库 `json`	`json` 默认返回 `dict / list`，不会自动得到带字段约束的 DTO
CSV	Apache Commons CSV 等	标准库 `csv`	两边都应交给解析器，不能用 `split(",")` 代替
对象序列化	`ObjectOutputStream`	`pickle`	都有反序列化风险，也都不适合作为开放的跨语言协议

路径和文件对象为什么要分开理解

`Path` 是文件系统地址的对象化表示，类似 Java 的 `java.nio.file.Path`：

```
from pathlib import Path

data_dir = Path("data")
users_path = data_dir / "users.json"
```

`/` 在这里是 `Path` 重载的路径拼接运算符。它会按当前操作系统的规则生成新路径，但不会创建目录、打开文件或访问磁盘。真正的 I/O 从 `open()`、`read_text()`、`write_text()` 等调用开始。

`open()` 或 `Path.open()` 返回文件对象。文件对象持有操作系统文件描述符，维护当前读写位置，并通常带有缓冲区，所以使用后必须关闭。

```
with users_path.open("r", encoding="utf-8") as file:
    content = file.read()
```

D2x6dg8fpjj0sff40000gp/T/crossnote-f-920bc95b64efe86140a18a842508.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

```
with users_path.open("r", encoding="utf-8") as file:
    content = file.read()

# 离开 with 后，file 已经关闭。
```

`with` 会调用上下文管理器的进入和退出逻辑。正常执行、`return`，甚至块内抛出异常，都会进入退出逻辑并关闭文件。这与 Java try-with-resources 解决的是同一个生命周期问题。

但 `with` 不能简单理解成 Python 版 `Closeable`。任何实现上下文管理协议的对象都能使用它，例如锁、数据库事务和临时目录。后续章节会进一步解释这个协议。

`Path.read_text()` 和 `Path.write_text()` 是适合小文件的便捷方法，它们内部会自行打开并关闭文件，因此外层不需要再写 `with`：

```
message_path = Path("message.txt")
message_path.write_text("你好，Python\n", encoding="utf-8")
message = message_path.read_text(encoding="utf-8")
```

打开模式决定会不会破坏已有数据

打开文件时应主动表达读写意图。尤其要警惕 `"w"`：它在成功打开已有文件后就会把原内容截断为零长度，而不是等 `write()` 成功后再覆盖。

模式	含义	文件不存在时	文件已存在时
`"r"`	读取文本	抛出 `FileNotFoundError`	从开头读取
`"w"`	覆盖写入文本	创建文件	立即清空原内容
`"a"`	追加文本	创建文件	从末尾写入
`"x"`	仅新建文本	创建文件	抛出 `FileExistsError`
`"rb"` / `"wb"`	读取/写入原始字节	取决于基础模式	不做文本编解码

例如，生成一个绝不能覆盖的初始化文件时，`"x"` 比“先检查是否存在，再用 `"w"` ”更可靠。后者在检查与打开之间存在竞态窗口：另一个进程可能恰好创建了同名文件。

文本模式处理 `str`，二进制模式处理 `bytes`。文本在磁盘上仍然是字节，因此读写两端必须对编码达成一致：

'2502x6dg8fpjj0sff40000gp/T/crossnote-f-920bc95b64efe86140a18a842508.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

文本模式处理 `str`，二进制模式处理 `bytes`。文本在磁盘上仍然是字节，因此读写两端必须对编码达成一致：

```
写入: str ——encode('utf-8')——> bytes
读取: bytes ——decode('utf-8')——> str
```

工程代码通常显式传入 `encoding="utf-8"`。省略它会采用环境默认编码，同一份代码换到其他操作系统、容器或终端环境后可能出现乱码或 `UnicodeDecodeError`。

小文件和大文件应该怎样读取

`read()` 与 `read_text()` 会把全部内容一次性放进内存，适合小型配置、模板和测试数据。文件大小不受控制时，应迭代文件对象，让业务代码一次处理一行，而不是积累整个文件：

```
from pathlib import Path

log_path = Path("application.log")
with log_path.open("r", encoding="utf-8") as file:
    for line_number, line in enumerate(file, start=1):
        content = line.removesuffix("\n")
        print(line_number, content)
```

文件对象内部仍会做缓冲读取，但业务代码不必同时保存所有行。需要注意，`strip()` 会删除行首和行尾的空格、制表符等字符；如果只想去掉读取到的换行，`removesuffix("\n")` 更不会误伤真实数据。

“逐行”也不是所有二进制大文件的答案。图片、压缩包或没有自然行边界的数据，通常使用 `file.read(chunk_size)` 分块处理。

文件错误应该在哪一层处理

`FileNotFoundError`、`PermissionError` 和 `UnicodeDecodeError` 表达的是不同失败原因。只有当前层知道如何恢复时才捕获异常：

```
from pathlib import Path

config_path = Path("optional.conf")
try:
    config = config_path.read_text(encoding="utf-8")
```

```python
from pathlib import Path

config_path = Path("optional.conf")
try:
    config = config_path.read_text(encoding="utf-8")
except FileNotFoundError:
    config = "mode=default\n"
```

这里的业务规则明确允许“可选配置不存在”，所以可以使用默认值。权限不足或内容编码损坏不是同一回事，不应被 `except Exception: pass` 一起伪装成“没有配置”。无法恢复时，让异常向上传播，或补充文件路径等上下文后继续抛出。

JSON 解决的是跨语言结构化交换

JSON 适合 Web API、配置文件和跨语言数据交换。Python 标准库自带 `json`；Java 8 标准库没有对应的完整 JSON API，实际项目通常使用 Jackson 或 Gson。

读取 JSON 后得到的是通用 Python 数据结构：

JSON 类型	Python 默认类型	需要注意
object	`dict`	JSON 对象的键只能是字符串
array	`list`	Python 元组写入后再读回会变成 `list`
string	`str`	文件编码通常约定为 UTF-8
number	`int / float`	金额不应随意使用二进制浮点数
true / false	`True / False`	Python 的拼写首字母大写
null	`None`	`null` 与字段完全缺失是两种状态

JSON 不携带 Python 类、集合、日期、时区或 `Decimal` 的业务语义。这些信息需要协议双方另外约定，例如 schema、ISO 8601 时间格式、金额的小数位数，以及是否允许未知字段。

`load`、`loads`、`dump`、`dumps` 怎么记

- `json.load(file)`：从文件对象读取 JSON；
- `json.loads(text)`：从字符串读取 JSON，名称末尾的 `s` 可以记成 string；
- `json.dump(data, file)`：把对象写入文件对象；

502x6dg8fpjj0sff40000gp/T/crossnote-f-920bc95b64efe86140a18a842508.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

`load` 、 `loads` 、 `dump` 、 `dumps` 怎么记

- `json.load(file)`：从文件对象读取 JSON；
- `json.loads(text)`：从字符串读取 JSON，名称末尾的 `s` 可以记成 string；
- `json.dump(data, file)`：把对象写入文件对象；
- `json.dumps(data)`：返回 JSON 字符串。

文件 API 与字符串 API 的差别只是输入输出位置，格式规则相同。

```
import json
from pathlib import Path

path = Path("settings.json")
with path.open("w", encoding="utf-8") as file:
    json.dump({"language": "中文"}, file, ensure_ascii=False, indent=2)
    file.write("\n")
```

`ensure_ascii=False` 让中文直接可读；`indent=2` 便于人工查看和版本比较。自定义对象不会自动变成稳定的 JSON 协议，通常应先挑选允许暴露的字段，转换成明确的 `dict` / `list` / 标量结构。

为什么“JSON 解析成功”不代表“数据可用”

处理外部 JSON 至少有两个独立阶段：

1. **语法解析**：括号、引号、逗号等是否符合 JSON 语法；失败时抛出 `JSONDecodeError`。
2. **业务校验**：顶层结构、必需字段、类型、取值范围、唯一性和未知字段策略是否符合约定。

下面的文本是合法 JSON，但 `True` 不能作为业务上的正整数用户 ID：

```
import json

user = json.loads('{"id": true, "name": "Alice"}')
user_id = user.get("id")

if type(user_id) is not int or user_id <= 0:
    raise ValueError("id 必须是正整数")
```

这里特意使用 `type(user_id) is int`，因为 Python 的 `bool` 是 `int` 的子类：`isinstance(True, int)` 会得到 `True`。对于允许整数子类的普通场景，`isinstance()` 仍是更惯用的写法；这里是严格边界校验的例外。

这里特意使用 `type(user_id) is int`，因为 Python 的 `bool` 是 `int` 的子类：`isinstance(True, int)` 会得到 `True`。对于允许整数子类的普通场景，`isinstance()` 仍是更惯用的写法；这里是严格边界校验的例外。

还要区分字段缺失和显式 `null`。`user.get("age")` 在两种情况下都会返回 `None`；如果协议认为它们含义不同，应先用 `"age" in user` 检查字段是否存在，再检查其值。

这和 Java 中“Jackson 成功反序列化”后仍需 Bean Validation 或领域校验是同一层次的问题。区别在于，Python 标准库默认给你的是 `dict`，不会借助 DTO 声明自动约束字段。

绝不要用 `eval()` 解析 JSON。它不遵循 JSON 的 `true`、`false`、`null` 语法，更严重的是它可能执行输入中的 Python 代码。

更新重要文件时怎样避免留下半成品

直接以 `"w"` 打开正式文件，会先清空旧内容。如果序列化失败、磁盘写满或进程中断，可能只留下半个新文件。

重要的小型配置通常采用下面的提交过程：

1. 在目标文件的同一目录创建临时文件
2. 把完整新内容写入临时文件
3. `flush()`，再用 `os.fsync()` 请求刷新文件内容
4. 关闭临时文件
5. 用 `os.replace()` 一次替换正式文件
6. 失败时清理遗留的临时文件

临时文件和目标文件放在同一目录，是为了让替换发生在同一文件系统；跨文件系统移动通常无法提供原子重命名。替换成功前，读者继续看到完整的旧版本；替换成功后，新打开文件的读者看到完整的新版本。

原子替换只解决“不要暴露半成品”，不等于完整事务：

- 两个写入者同时更新时通常仍是最后一次替换获胜，需要锁或更高层的版本检查；
- 它不负责 schema 校验、备份、权限和磁盘空间管理；
- 对断电后的严格持久性有更高要求时，还要考虑所在操作系统和文件系统对目录元数据同步的保证。

pickle 为什么只能加载可信文件

pickle 是 Python 专用的二进制对象序列化格式。它能保留集合、元组以及更复杂的 Python 对象图，读写时必须使用二进制模式：

```python
import pickle
from pathlib import Path

cache_path = Path("cache.pickle")
with cache_path.open("wb") as file:
    pickle.dump({"ids": {1, 2}}, file)

with cache_path.open("rb") as file:
    cached = pickle.load(file)  # 仅限自己生成且可信的文件
```

`pickle.load()` 不是单纯读取数据。pickle 协议可以指示 Python 导入并调用对象，恶意内容可能借此执行任意代码。因此，用户上传、邮件附件、网络响应和来源不明的缓存都不能反序列化。文件扩展名、异常捕获和“先看看内容”都不能消除这个风险。

这可以类比 Java 原生对象反序列化的安全问题。pickle 适合可删除并重建的可信短期缓存或实验中间结果，不适合跨语言协议和长期归档；类名、模块路径或代码版本变化都可能令旧文件失效。

CSV 为什么不能用 `split(",")`

CSV 用于二维表格交换，但“逗号分隔”只是表面描述。字段本身可以包含逗号、双引号甚至换行：

```
id,name,note
2,"Doe, Jane","喜欢 ""Python""
也喜欢文件处理"
```

第二条逻辑记录跨了两条物理文本行，但仍然只有三个字段。`split(",")` 不理解引号和转义，一定会在这类输入上错位；应使用标准库 `csv` 。

打开 CSV 时要传入 `newline=""` ，让 `csv` 模块统一识别和生成换行，避免文本层与 CSV 层重复处理换行：

```python
import csv
from pathlib import Path

path = Path("users.csv")
with path.open("r", encoding="utf-8", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        user_id = int(row["id"])
        active_text = row["active"].strip().lower()
```

```
import csv
from pathlib import Path

path = Path("users.csv")
with path.open("r", encoding="utf-8", newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        user_id = int(row["id"])
        active_text = row["active"].strip().lower()
        if active_text not in {"true", "false"}:
            raise ValueError(f"第 {reader.line_num} 行 active 无效")
        active = active_text == "true"
```

`DictReader` 用表头作为键，比按位置访问更容易维护，但它不会自动完成 schema 和业务校验。读取边界上还应处理：

- 所有正常字段最初都是字符串， `bool("False")` 仍会得到 `True` ，所以布尔值必须显式解析；
- 表头可能缺失、重复、顺序不符或包含未知字段；
- 缺少的字段值可能是 `None` ，多余字段默认会放到键 `None` 对应的列表中；
- `int()` 等类型转换可能抛出 `ValueError` ，错误信息应带上输入位置；
- ID 唯一性、年龄范围等跨字段或跨记录规则仍要交给业务校验。

`reader.line_num` 表示解析器已经读取到的**物理行号**。当一个带换行的字段跨越多行时，它通常指向当前逻辑记录结束所在的物理行，而不是“第几条数据记录”。错误信息中可以使用它定位源文件，但不要把它当作记录计数器。

如果 CSV 会被 Excel 等电子表格软件打开，还要单独评估公式注入：以 `=` 、 `+` 、 `-` 、 `@` 开头的不可信文本可能被消费端当成公式。这不是 CSV 引号规则本身能够解决的问题。

临时目录适合什么场景

示例、测试和可重建的中间计算不应污染源码目录。 `TemporaryDirectory` 也是上下文管理器：

```
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory(prefix="python-files-") as directory:
    root = Path(directory)
    (root / "result.txt").write_text("临时结果", encoding="utf-8")
    print(root.exists())  # True
```

```python
import tempfile
from pathlib import Path

with tempfile.TemporaryDirectory(prefix="python-files-") as directory:
    root = Path(directory)
    (root / "result.txt").write_text("临时结果", encoding="utf-8")
    print(root.exists())  # True

print(root.exists())      # False
```

退出 `with` 时会递归清理目录，即使块内发生异常也会尝试清理。不要把需要长期保存的数据放入系统临时目录，因为操作系统或清理任务可以随时删除它。

如何选择存储格式

场景	建议起点	优势	主要风险
人类阅读的纯文本	UTF-8 文本	简单、可搜索、 方便版本比较	编码、换行、大文件内存占用
Web API 与跨语言数据	JSON + schema 校验	生态广、语言边界清楚	类型信息有限、精度、 只解析不校验
表格导入导出	CSV + 明确方言/schema	办公和数据工具支持广	引号、字段内换行、 原始字段都是字符串
Python 可信短期缓存	pickle（谨慎）	可保存复杂 Python 对象	不可信输入可执行代码、 版本耦合
大规模查询与并发更新	数据库	查询、 事务和并发控制更合适	运维与 schema 迁移成本

不要只按“哪种格式代码最短”来选择。先回答：谁负责读取、是否跨语言、数据保存多久、输入是否可信、规模有多大、失败后如何恢复，以及是否需要多个进程并发查询或更新。

Java 工程师最容易踩的坑

- 把 `Path` 当成已打开资源：构造路径不会访问磁盘，真正需要关闭的是文件对象。
- 忘记 `with`：异常路径可能泄漏文件描述符，缓冲内容也可能没有及时写出。

Java 工程师最容易踩的坑

- 把 `Path` 当成已打开资源：构造路径不会访问磁盘，真正需要关闭的是文件对象。
- 忘记 `with`：异常路径可能泄漏文件描述符，缓冲内容也可能没有及时写出。
- 省略 `encoding`：换一台机器后，同一文件可能解码失败或产生乱码。
- 用 `"w"` 直接更新重要配置：打开文件时旧内容已经被清空，应评估原子替换。
- 对不受控的大文件直接 `read()`：内存占用随文件增长，应逐行或分块处理。.
- 把 JSON 解析成功当作 DTO 校验成功：合法 JSON 仍可能缺字段、类型错误或违反领域规则。
- 用 `eval()` 读 JSON，或用 `pickle.load()` 读外部文件：两者都可能执行不可信代码。
- 用 `split(",")` 读 CSV：带逗号、引号或换行的字段会被错误拆分。
- 用 `bool(row["active"])` 转换 CSV 布尔值：任何非空字符串都会得到 `True`。
- 把原子替换当成并发锁：它不阻止多个写入者相互覆盖。

本章小结

文件是程序与持久化世界之间的边界。`Path` 负责表达位置，文件对象负责 I/O，`with` 负责资源生命周期，编码负责 `str` 与 `bytes` 的转换，JSON/CSV/pickle 负责不同的数据格式。格式解析之后仍要进行类型转换和业务校验；重要文件写入时还要设计失败策略。

Python 的标准库让常见文件操作比 Java 原型代码更短，但工程责任没有减少。编码、schema、安全边界、异常传播、并发写入和恢复策略仍然需要显式设计。

运行本章示例

项目配置要求 Python 3.14 或更高版本；本章只使用标准库，没有第三方依赖。配套文件是：

- `example.py`：完整演示文本、JSON 原子写入、pickle、CSV、业务校验和临时目录清理；
- `Chapter10Files.java`：提供 Java 8 的资源管理和格式处理对照。Java 文件中的 JSON/CSV 拼接只服务于受控教学输入，生产代码应使用成熟库。

推荐在工程根目录（即包含 `pyproject.toml` 的目录）使用统一运行器，它会把 Java `.class` 编译到临时目录：

```
# 同时运行 Python 和 Java 示例
python3 java_to_python/run_all.py --chapter 10

# 没有安装 JDK 时，只运行 Python 示例
python3 java_to_python/run_all.py --chapter 10 --python-only
```

Python 的标准库让常见文件操作比 Java 原型代码更短，但工程责任没有减少。编码、schema、安全边界、异常传播、并发写入和恢复策略仍然需要显式设计。

运行本章示例

项目配置要求 Python 3.14 或更高版本；本章只使用标准库，没有第三方依赖。配套文件是：

- `example.py`：完整演示文本、JSON 原子写入、pickle、CSV、业务校验和临时目录清理；
- `Chapter10Files.java`：提供 Java 8 的资源管理和格式处理对照。Java 文件中的 JSON/CSV 拼接只服务于受控教学输入，生产代码应使用成熟库。

推荐在工程根目录（即包含 `pyproject.toml` 的目录）使用统一运行器，它会把 Java `.class` 编译到临时目录：

```
# 同时运行 Python 和 Java 示例
python3 java_to_python/run_all.py --chapter 10

# 没有安装 JDK 时，只运行 Python 示例
python3 java_to_python/run_all.py --chapter 10 --python-only
```

也可以直接运行 Python 文件：

```
python3 java_to_python/chapters/10_files/example.py
```

运行后重点观察："x" 如何拒绝覆盖已有文件；JSON 序列化失败时旧文件是否保留；JSON 语法错误与业务错误的异常有何不同；pickle 如何保留 `set` / `tuple`；CSV 的物理行数为何大于逻辑记录数；离开 `TemporaryDirectory` 后目录是否消失。

动手练习

1. 创建三行文本，分别用 `read_text()` 和逐行迭代读取。比较返回对象、换行处理和内存使用方式。
2. 给用户数据增加 `age` 字段，要求它是 `0..150` 的整数。分别测试字段缺失、`true`、负数和字符串输入。
3. 手工截断 JSON 文件，捕获 `JSONDecodeError`，输出文件名、行号和列号，但不要静默返回空列表。
4. 给 CSV 增加一个同时包含逗号、双引号和换行的备注。比较物理文本行与 `DictReader` 产生的逻辑记录，再让非法布尔值的错误带上源文件行号。
5. 把原子 JSON 写入封装为接收任意目标路径的配置保存函数。再思考两个进程同时更新时，“最后一次替换获胜”是否满足业务；若不满足，应在哪一层增加并发控制？

```python
"""第 10 章：安全地读写文本、JSON、pickle 与 CSV。"""

import csv
import json
import os
import pickle
import tempfile
from pathlib import Path


USERS: list[dict[str, object]] = [
    {"id": 1, "name": "Alice", "age": 28, "active": True, "note": "普通字段"},
    {
        "id": 2,
        "name": "Doe, Jane",
        "age": 35,
        "active": False,
        "note": '喜欢 "Python"\n也喜欢文件处理',
    },
]
USER_FIELDS = ("id", "name", "age", "active", "note")


def print_section(title: str) -> None:
    """用清楚的标题分隔终端输出。"""
    print(f"\n=== {title} ===")


def demonstrate_text_files(root: Path) -> None:
    """演示 UTF-8、覆盖/追加/仅新建模式，以及逐行读取。"""
    print_section("1. 文本文件与资源管理")
    text_path: Path = root / "notes.txt"

    # write_text 适合一次写入不大的文本；返回值是写入的字符数。
    initial_text = "第一行：你好\n第二行：Python\n"
    written_count: int = text_path.write_text(initial_text, encoding="utf-8")

```

```python
def demonstrate_text_files(root: Path) -> None:

    # "a" 不会清空已有内容。with 确保正常或异常退出时都会关闭文件。
    with text_path.open("a", encoding="utf-8") as file:
        file.write("第三行：追加内容\n")

    # 文件对象本身是可迭代对象，适合逐行处理大文件。
    numbered_lines: list[str] = []
    with text_path.open("r", encoding="utf-8") as file:
        for line_number, line in enumerate(file, start=1):
            # 不使用 strip()，以免意外删除属于数据的行首、行尾空格。
            content: str = line.removesuffix("\n")
            numbered_lines.append(f"{line_number}: {content}")

    print("首次写入字符数:", written_count)
    print("逐行读取:", numbered_lines)

    # "x" 表达“只能新建”：目标已存在时明确失败，不会误覆盖。
    try:
        with text_path.open("x", encoding="utf-8") as file:
            file.write("这行不会执行")
    except FileExistsError:
        print("x 模式保护已有文件：FileExistsError")

    # 只在当前层能解释或恢复时捕获具体异常。
    try:
        (root / "missing.txt").read_text(encoding="utf-8")
    except FileNotFoundError as error:
        print("缺少可选文件，使用默认值:", error.filename is not None)


def validate_users(data: object) -> list[dict[str, object]]:
    """把解析结果变成经过业务规则校验的用户列表。"""
    if not isinstance(data, list):
        raise ValueError("顶层 JSON 必须是数组")

    expected_fields: set[Literal['id', 'name', 'age', 'acti... = set(USER_FIELDS)
```

```python
def validate_users(data: object) -> list[dict[str, object]]:
    raise ValueError("顶层 JSON 必须是数组")

    expected_fields: set[Literal['id', 'name', 'age', 'acti... = set(USER_FIELDS)
    seen_ids: set[int] = set()
    validated_users: list[dict[str, object]] = []

    for position, item in enumerate(data, start=1):
        if not isinstance(item, dict):
            raise ValueError(f"第 {position} 个用户必须是对象")

        actual_fields: set[Any] = set(item)
        if actual_fields != expected_fields:
            missing: list[Literal['id', 'name', 'age', 'act... = sorted(expected_fields - actual_fie
            extra: list[Any] = sorted(actual_fields - expected_fields)
            details: str = f"missing={missing}, extra={extra}"
            raise ValueError(f"第 {position} 个用户字段不符: {details}")

        user_id: Any = item["id"]
        name: Any = item["name"]
        age: Any = item["age"]
        active: Any = item["active"]
        note: Any = item["note"]

        # bool 是 int 的子类; type(...) is int 才不会把 True 当成 ID 1。
        if type(user_id) is not int or user_id <= 0:
            raise ValueError(f"第 {position} 个用户 id 必须是正整数")
        if not isinstance(name, str) or not name.strip():
            raise ValueError(f"第 {position} 个用户 name 必须是非空字符串")
        if type(age) is not int or not 0 <= age <= 150:
            raise ValueError(f"第 {position} 个用户 age 必须是 0 到 150 的整数")
        if type(active) is not bool:
            raise ValueError(f"第 {position} 个用户 active 必须是布尔值")
        if not isinstance(note, str):
            raise ValueError(f"第 {position} 个用户 note 必须是字符串")
        if user_id in seen_ids:
            raise ValueError(f"用户 id 重复: {user_id}")
```

```python
def validate_users(data: object) -> list[dict[str, object]]:

        seen_ids.add(user_id)
        validated_users.append(
            {"id": user_id, "name": name, "age": age, "active": active, "note": note}
        )

    return validated_users


def atomic_write_json(path: Path, data: object) -> None:
    """在目标的同一目录写完临时文件，再原子替换目标文件。"""
    temporary_path: Path | None = None
    try:
        # delete=False 让文件关闭后仍存在，随后才能用 replace() 移动它。
        with tempfile.NamedTemporaryFile(
            "w",
            encoding="utf-8",
            newline="\n",
            prefix=f".{path.name}.",
            suffix=".tmp",
            dir=path.parent,
            delete=False,
        ) as file:
            temporary_path = Path(file.name)
            json.dump(data, file, ensure_ascii=False, indent=2)
            file.write("\n")
            file.flush()  # 把 Python 用户态缓冲区交给操作系统。
            os.fsync(file.fileno())  # 请求操作系统把该文件内容刷新到存储设备。

        # 临时文件与目标在同一文件系统，replace 不会暴露半成品。
        temporary_path.replace(path)
    finally:
        # 成功替换后原临时路径已不存在；失败时则在这里清理半成品。
        if temporary_path is not None:
            temporary_path.unlink(missing_ok=True)

```

```python
def demonstrate_json(root: Path) -> None:
    """演示 JSON 原子写入、语法错误与业务校验错误。"""
    print_section("2. JSON: 解析、校验与原子替换")
    json_path: Path = root / "users.json"

    # 模拟已经存在的旧配置；正式写入不会先用 "w" 清空它。
    json_path.write_text('{"version": "old"}\n', encoding="utf-8")
    atomic_write_json(json_path, USERS)

    with json_path.open("r", encoding="utf-8") as file:
        raw_data: Any = json.load(file)
    loaded_users: list[dict[str, object]] = validate_users(raw_data)
    json_text: str = json_path.read_text(encoding="utf-8")
    print("JSON 校验后往返相等:", loaded_users == USERS)
    print("UTF-8 中文仍可直接阅读:", "普通字段" in json_text)

    # set 不能写入 JSON。失败发生在临时文件中，不会破坏正式文件。
    try:
        atomic_write_json(json_path, {"unsupported": {1, 2}})
    except TypeError:
        leftover_files: list[Path] = list(root.glob(f".{json_path.name}.*.tmp"))
        target_unchanged: bool = json_path.read_text(encoding="utf-8") == json_text
        safe_failure: bool = target_unchanged and not leftover_files
        print("写入失败保留旧文件并清理临时文件:", safe_failure)

    # 语法错误由 JSON 解析器报告行列；它和业务规则错误是两类问题。
    try:
        json.loads('{"id": 1,}')
    except json.JSONDecodeError as error:
        print(f"坏 JSON 位置: 第 {error.lineno} 行，第 {error.colno} 列")

    # 这是合法 JSON/Python 数据，但 True 不能充当业务上的整数 ID。
    invalid_users: list[dict[str, Any]] = [
        {"id": True, "name": "Bob", "age": 30, "active": True, "note": "反例"}
    ]
    try:
```

```python
def demonstrate_json(root: Path) -> None:
    try:
        validate_users(invalid_users)
    except ValueError as error:
        print("业务校验拒绝合法但无效的数据:", error)


def demonstrate_pickle(root: Path) -> None:
    """只加载本程序刚写出的可信 pickle，并展示其 Python 专用特征。"""
    print_section("3. pickle：仅限可信 Python 数据")
    pickle_path: Path = root / "cache.pickle"
    python_only_data: dict[str, Any] = {
        "users": USERS,
        "favorite_ids": {1, 2},
        "point": (3, 4),
    }

    with pickle_path.open("wb") as file:
        pickle.dump(python_only_data, file, protocol=pickle.HIGHEST_PROTOCOL)
    with pickle_path.open("rb") as file:
        # 安全边界：该文件由当前进程刚写出，不接受外部内容。
        loaded_data: Any = pickle.load(file)

    print("pickle 往返相等:", loaded_data == python_only_data)
    print("pickle 保留 set/tuple:", type(loaded_data["favorite_ids"]).__name__, end=" / ")
    print(type(loaded_data["point"]).__name__)

    # JSON 没有 tuple 类型，因此元组会退化为 list。
    json_point: Any = json.loads(json.dumps({"point": (3, 4)}))
    print("JSON 中 tuple 读回类型:", type(json_point["point"]).__name__)
    print("安全提醒：永远不要 pickle.load() 不可信文件")


def parse_csv_boolean(value: str, *, line_number: int) -> bool:
    """显式解析 CSV 布尔文本，避免 bool('False') 仍为 True。"""
    normalized: str = value.strip().lower()
```

```python
def parse_csv_boolean(value: str, *, line_number: int) -> bool:
    """显式解析 CSV 布尔文本，避免 bool('False') 仍为 True。"""
    normalized: str = value.strip().lower()
    if normalized == "true":
        return True
    if normalized == "false":
        return False
    raise ValueError(f"CSV 第 {line_number} 行布尔值无效: {value!r}")


def parse_csv_user(row: dict[str, str | None], *, line_number: int) -> dict[str, object]:
    """把一行 CSV 字符串转换成业务类型，并给错误附加行号。"""
    if len(row) != len(USER_FIELDS):
        raise ValueError(f"CSV 第 {line_number} 行存在多余字段")

    missing_values: list[str] = [field for field in USER_FIELDS if row.get(field) is None]
    if missing_values:
        raise ValueError(f"CSV 第 {line_number} 行缺少值: {missing_values}")

    # 上面的检查已排除 None；局部变量和 assert 让运行时前提更清楚。
    id_text: str | None = row["id"]
    name: str | None = row["name"]
    age_text: str | None = row["age"]
    active_text: str | None = row["active"]
    note: str | None = row["note"]
    assert id_text is not None and name is not None and age_text is not None
    assert active_text is not None and note is not None

    try:
        user_id = int(id_text)
    except ValueError as error:
        raise ValueError(f"CSV 第 {line_number} 行 id 不是整数: {id_text!r}") from error

    try:
        age = int(age_text)
    except ValueError as error:
        raise ValueError(f"CSV 第 {line_number} 行 age 不是整数: {age_text!r}") from error
```

```
220  def parse_csv_user(row: dict[str, str | None], *, line_number: int) -> dict[str, object]:
242
243      try:
244          age = int(age_text)
245      except ValueError as error:
246          raise ValueError(f"CSV 第 {line_number} 行 age 不是整数: {age_text!r}") from error
247
248      return {
249          "id": user_id,
250          "name": name,
251          "age": age,
252          "active": parse_csv_boolean(active_text, line_number=line_number),
253          "note": note,
254      }
255
256
257  def demonstrate_csv(root: Path) -> None:
258      """演示 CSV 引号规则、原始字符串类型和显式类型转换。"""
259      print_section("4. CSV: 特殊字符与类型转换")
260      csv_path: Path = root / "users.csv"
261
262      with csv_path.open("w", encoding="utf-8", newline="") as file:
263          writer: DictWriter[Literal['id', 'name', 'age'... = csv.DictWriter(file, fieldnames=USER_FIELDS
264          writer.writeheader()
265          writer.writerows(USERS)
266
267      physical_lines: list[str] = csv_path.read_text(encoding="utf-8").splitlines()
268      line_with_comma: str = next(line for line in physical_lines if "Doe" in line)
269      print("错误 split(',') 得到字段数:", len(line_with_comma.split(",")), "(实际应为 5)")
270      print("字段内换行使物理行数:", len(physical_lines), "(逻辑记录含表头为 3)")
271
272      converted_users: list[dict[str, object]] = []
273      raw_types: dict[str, str] | None = None
274      with csv_path.open("r", encoding="utf-8", newline="") as file:
275          reader: DictReader[str] = csv.DictReader(file)
276          if reader.fieldnames != list(USER_FIELDS):
277              raise ValueError(f"CSV 表头不符: {reader.fieldnames}")
```

```
257 def demonstrate_csv(root: Path) -> None:
274     with csv_path.open("r", encoding="utf-8", newline="") as file:
275         reader: DictReader[str] = csv.DictReader(file)
276         if reader.fieldnames != list(USER_FIELDS):
277             raise ValueError(f"CSV 表头不符: {reader.fieldnames}")
278 
279         for row in reader:
280             if raw_types is None:
281                 raw_types = {field: type(row[field]).__name__ for field in USER_FIELDS}
282             converted_users.append(parse_csv_user(row, line_number=reader.line_num))
283 
284     # 格式层负责字符串转换，统一业务校验仍由 validate_users 负责。
285     loaded_users: list[dict[str, object]] = validate_users(converted_users)
286     print("CSV 原始字段类型:", raw_types)
287     print("CSV 转换并校验后往返相等:", loaded_users == USERS)
288 
289     try:
290         parse_csv_boolean("yes", line_number=99)
291     except ValueError as error:
292         print("坏 CSV 值包含定位信息:", error)
293 
294 
295 def demonstrate_readme_exercises(root: Path) -> None:
296     """运行 README 中关于读取策略、年龄校验和异常路径的练习。"""
297     print_section("5. README 小练习")
298 
299     print("练习 1: 一次读取与逐行读取")
300     text_path: Path = root / "exercise-notes.txt"
301     text_path.write_text("第一行\n第二行\n第三行\n", encoding="utf-8")
302     complete_text: str = text_path.read_text(encoding="utf-8")
303     with text_path.open("r", encoding="utf-8") as file:
304         streamed_lines: list[str] = [line.removesuffix("\n") for line in file]
305     print("  read_text 返回类型/长度:", type(complete_text).__name__, len(complete_text))
306     print("  逐行读取返回:", streamed_lines)
307     print("  两种方式内容一致:", complete_text.splitlines() == streamed_lines)
308 
```

```python
def demonstrate_readme_exercises(root: Path) -> None:

    print("练习 2: 年龄字段的四种错误输入")
    valid_user: dict[str, Any] = {
        "id": 3,
        "name": "Cara",
        "age": 20,
        "active": True,
        "note": "年龄校验",
    }
    invalid_age_cases: dict[str, Any] = {
        "缺字段": {key: value for key, value in valid_user.items() if key != "age"},
        "布尔值": valid_user | {"age": True},
        "负数": valid_user | {"age": -1},
        "字符串": valid_user | {"age": "20"},
    }
    for label, invalid_user in invalid_age_cases.items():
        try:
            validate_users([invalid_user])
        except ValueError as error:
            print(f"  {label}:", error)

    print("练习 3: 截断 JSON 的文件名、行号与列号")
    truncated_path: Path = root / "truncated.json"
    truncated_path.write_text('[\n  {"id": 1,\n', encoding="utf-8")
    try:
        with truncated_path.open("r", encoding="utf-8") as file:
            json.load(file)
    except json.JSONDecodeError as error:
        print(
            f"  {truncated_path.name}: 第 {error.lineno} 行, 第 {error.colno} 列 - {error.msg}"
        )

    print("练习 4: CSV 的特殊字段、物理行和坏布尔值结果见上方第 4 节")

    print("练习 5: 连续原子替换时最后一次写入获胜")
    config_path: Path = root / "exercise-config.json"
```

```
295 def demonstrate_readme_exercises(root: Path) -> None:
341
342     print("练习 5: 连续原子替换时最后一次写入获胜")
343     config_path: Path = root / "exercise-config.json"
344     atomic_write_json(config_path, {"writer": "first"})
345     atomic_write_json(config_path, {"writer": "second"})
346     with config_path.open("r", encoding="utf-8") as file:
347         final_config: Any = json.load(file)
348     print("  最终配置:", final_config)
349     print("  原子替换不等于并发锁；多写入者仍需额外协调")
350
351
352 def main() -> None:
353     """在自动清理的目录中依次运行所有示例。"""
354     with tempfile.TemporaryDirectory(prefix="python-files-") as directory:
355         root = Path(directory)
356         demonstrate_text_files(root)
357         demonstrate_json(root)
358         demonstrate_pickle(root)
359         demonstrate_csv(root)
360         demonstrate_readme_exercises(root)
361         print_section("6. 临时目录")
362         print("示例运行期间目录存在:", root.exists())
363
364     print("退出 with 后目录已清理:", not root.exists())
365
366
367 if __name__ == "__main__":
368     main()
369
```